LiveRecall Report

Ow Jun Duan (A0308148A) · Ng Yu Hao Benedict (A0299879N) · Max Khoo Zheng Ning (A0307430R) · Tan Yi Qian (A0301651U)

Abstract

LiveRecall is a browser-based classroom companion that distributes revision-based captions to concurrent readers and restores missed finalized text after interruptions. It combines monotonic revision state, local 15/30/60-second recall, bounded history and independent adaptive delivery channels. In three-run Ubuntu tests, 100 normal clients achieved 32.7ms P95 caption age with 100% final completeness. Under 10% slow readers, adaptive delivery reduced overall P95 from 2,252.9 to 31.2ms versus coalescing while raising final completeness from 95.13% to 100%. All reported T1–T5 workloads now achieved 100% final completeness, combined delay/loss nevertheless reduced fresh-final delivery to 89.49%, and adaptive scheduling increased latency under the 2 Mbit/s profile.

1 Motivation and requirements

Students who briefly lose track of a lecture need the latest words and the preceding minute, not an unbounded transcript. A caption may change several times before becoming final. The main networking problem is therefore to distribute successive revisions, preserve final content, isolate slow receivers, and recover after reconnects without replaying old or duplicate state. The deterministic caption_source.py is used, speech recognition is not part of the mandatory path.

2 Architecture and application protocol

END-TO-END DATA FLOW

caption_source.py → SOURCE_HELLO / CAPTION_UPDATE → LiveRecallHub → independent ClientChannel writers → browser CaptionStore → transcript and local recall

The asynchronous Python WebSocket server separates source ingestion, authoritative per-session state, and outbound client delivery. After SOURCE_HELLO, the source emits CAPTION_UPDATE with session, segment, revision, source_ts, final and text. A student or load client sends CLIENT_HELLO (client_id, session, last_segment, last_revision, history_seconds), receives an ordered HISTORY snapshot of missing finals, and then consumes live CAPTION_UPDATE frames.

Protocol frame	Direction	Role
SOURCE_HELLO	Source → server	Select caption session (e.g., L1)
CAPTION_UPDATE	Source → server → clients	Revision and durable final-state propagation
CLIENT_HELLO	Client → server	Join or resume using final-state cursor
HISTORY	Server → client	Only recent finalized state not already held

Implementation: solution_server.py → liverecall/hub.py, caption_state.py, client_channel.py, flow_control.py, semantic_queue.py, browser modules reside in client/. Protocol fields follow the assignment compatibility interface.

3 Revision correctness, local recall and recovery

3.1 Monotonic and final-state semantics

CaptionState accepts a newer revision for an active segment and rejects stale/equal revisions, once `final=true` is accepted, subsequent updates for that segment are rejected. The browser CaptionStore applies the same checks and renders one row per segment, replacing provisional text in place rather than appending duplicates. Source revisions remain visible under normal conditions, a final supersedes all pending provisional state for that particular segment.

3.2 Bounded state, reconnect and late join

The server retains recent finalized captions for 90 seconds, capped at 4,096 items, and at most 64 active provisional segments. Each browser retains up to 90 seconds / 1,000 segments for local recall. “What did I miss?” initially shows 30 seconds and offers 15 and 60 seconds without requesting the server during normal operation. After a disconnect, the browser automatically reconnects and resends its most recent finalized (segment, revision) cursor, the server returns newer finalized items within the requested history window (maximum 60 seconds). A new client uses cursor (0, 0), HISTORY and live updates merge into the same deduplicating store. The channel is registered before yielding after the history snapshot, avoiding a history/live handoff gap.

CORRECTNESS INVARIANTS

No older revision overwrites a newer one, no finalized segment is overwritten or displayed twice, recovery HISTORY contains finalized entries only and is ordered by segment. Recovery is bounded by retained server history, not an indefinitely growing transcript.

4 Backpressure and overload strategy

The submitted `--policy` adaptive design decouples source intake from each client: the hub offers each accepted update to that client’s semantic queue, while an independent writer task sends to its own WebSocket. A blocked sender cannot stall source ingestion or other writers. During NORMAL operation, the queue preserves all useful revisions. Lag is inferred from an unanswered WebSocket ping exceeding the minimum observed RTT by 0.3s, or from a transport write backlog above 64 KiB. This detects receiver/path lag earlier than waiting for `send()` to block.

On congestion, the controller drains outstanding transport backlog, collapses superseded provisional revisions to the newest per segment, prioritizes final captions when the drain is clear, holds provisional-only state until it has waited at least 1s, and paces traffic starting at 0.5s per frame (backing off up to 4s). It subsequently probes for recovery and returns to NORMAL when lag has subsided. These choices preserve durable final state while spending less network capacity on revisions that will soon be superseded.

Bound / fail-safe	Default	Consequence
Pending segments per client	256	Overflow triggers WebSocket close 1013, client reconnects for HISTORY
Pending revisions per segment	16	Oldest provisional can be removed when this cap is reached
Delivered-state memory per client	512 segments	Deduplication record is bounded
History / active state / sessions	4,096 finals / 64 active / 16 sessions	Time and count caps constrain retained application state

Comparison controls: FIFO intentionally queues every revision without a bound, coalesce collapses pending superseded revisions but lacks adaptive lag detection. The FIFO mode is a benchmark baseline, not the submitted default.

5 Public experiments and measurement method

Measurements were obtained on Ubuntu using the unchanged deterministic source and public load-test analyser. T1 used 20 clients/20s, T2 used 100 clients/40s, T3 used 100 clients/60s including 10% readers delayed by 500ms, T4 used 100 clients/60s including 20% disconnected after 15s for 5s, and T5 used 100 clients/40s with tc/netem or tbf on loopback (lo). Each scenario was run independently three times. Every value shown is the median of the three per-run statistics, not a percentile recomputed from pooled observations.

Caption age is client receive time minus source_ts, P50/P95/P99 characterize its distribution among accepted live updates and exclude browser paint time. “Fresh” means accepted live messages within 500ms, fresh-final instead uses the first correct delivery of each generated final, including recovered finals. Completeness counts unique finalized segments received over all expected client–segment pairs. Update coverage counts unique live revisions against generated revision–client pairs. These denominators differ, so a high live freshness rate need not imply high final freshness or full coverage.

5.1 Required T1–T5 results

Workload	P50 ms	P95 ms	P99 ms	Fresh	Fresh final	Final (%)	Coverage
T1 · basic (20)	5.7	11.4	15.1	100.00%	100.00%	100.00%	100.00%
T2 · scale (100)	16.4	32.7	47.3	100.00%	100.00%	100.00%	100.00%
T3 · 10% slow	15.3	36.6	6,111.9	96.89%	94.08%	100.00%	94.39%
T4 · reconnect	12.7	28.8	43.0	100.00%	98.42%	100.00%	98.42%
T5 · 2 Mbit/s	27.7	52.8	63.5	100.00%	100.00%	100.00%	100.00%
T5 · delay	207.8	256.7	276.9	100.00%	100.00%	100.00%	100.00%
T5 · loss	17.2	203.8	230.4	99.99%	99.96%	100.00%	98.64%
T5 · delay + loss	260.1	610.5	789.0	91.40%	89.49%	100.00%	58.05%
T5 · reorder	60.5	76.3	81.8	100.00%	100.00%	100.00%	100.00%

Table 1. Three-run median client-receive caption ages (ms) and delivery ratios. All 15 reported scenarios recorded zero observed duplicate and stale messages. Coverage denotes unique live revisions received out of expected source revision-client pairs.

5.2 Interpretation: scale, slow readers and recovery

T2 delivered all 14,700 expected live revisions and 4,900/4,900 final deliveries (P95 32.7ms). In T3, all 7,600/7,600 finals arrived despite 94.39% update coverage: normal-client P95 was 29.8ms, versus 8,190.2ms for slow clients, driving overall P99 to 6,111.9ms. Under T4, 20 reconnecting clients received 120 HISTORY items per run in aggregate. All 7,600 finals were received, with no duplicate or stale messages, recovery P95 was 8.0ms. Fresh-final delivery remained 98.42% because a recovered final can arrive correctly but too late to count as fresh.

5.3 Linux impairment behaviour

The captured qdiscs apply 2 Mbit/s TBF (4 KiB burst, 400ms latency), 100 ± 20ms delay, 3% loss, combined 120 ± 30ms delay/3% loss and 20ms reordering. With 2 Mbit/s shaping, P95 was 52.8ms and all revisions/finals arrived. Loss alone raised P95 to 203.8ms (P99 230.4ms), consistent with variable TCP retransmission delay. Combined delay/loss gave P95 610.5ms, 89.49% fresh finals and 58.05% coverage, but 4,900/4,900 finals ultimately arrived within the test window. Thus TCP/WebSocket reliability preserved final state here without guaranteeing timely revision delivery.

6 Before/after optimization evaluation

The public slow-reader workload was repeated under three `--policy` settings: **FIFO** (basic comparator), **coalesce** (revision suppression without adaptive lag control), and **adaptive** (submitted design). The tables report medians across three independent runs per mode. Workload settings were matched, without an embedded tested-commit fingerprint, stronger attribution beyond these controlled comparisons is limited.

6.1 T3: slow-reader stress (100 clients, 10 slow)

Mode	P50 ms	P95 ms	P99 ms	Fresh final	Final comp.	Normal P95	Slow P95
FIFO	9.84	2,255.44	24,028.16	90.00%	95.13%	16.06	28,079.53
Coalesce	9.83	2,252.87	24,030.26	90.00%	95.13%	16.27	28,093.25
Adaptive	14.38	31.19	6,113.81	94.08%	100.00%	26.16	8,184.05

Table 2. T3 policy comparison: P50/P95/P99 are overall caption ages, while normal/slow P95 are client-group-specific. Final completeness uses all expected client-final pairs.

Relative to coalescing, adaptive reduced overall P95 by 98.62% (2,252.87 to 31.19ms), P99 by 74.56% (24,030.26 to 6,113.81ms) and slow-reader P95 by 70.87% (28,093.25 to 8,184.05ms). Final completeness increased from 7,230/7,600 (95.13%) to 7,600/7,600 (100%), and fresh-final delivery rose from 90.00% to 94.08%. This gain came with lower revision coverage (94.39% versus 95.18%) and higher normal-client P95 (26.16 versus 16.27ms). The policy substantially improved the slow-client tail but did not eliminate multi-second delays.

6.2 T5: 2 Mbit/s constrained-bandwidth comparison

Mode	P50 ms	P95 ms	P99 ms	Fresh final	Final comp.	Update cover.
FIFO	14.44	37.58	41.29	100.00%	100.00%	100.00%
Coalesce	14.63	37.84	41.26	100.00%	100.00%	100.00%
Adaptive	26.47	50.82	57.33	100.00%	100.00%	100.00%

Table 3. Three-run medians from the 100-client, 40-second policy comparison under the same 2 Mbit/s TBF network profile.

Adaptive P95 (50.82 ms) was 35.23% higher than FIFO (37.58 ms), although all three policies achieved 100% freshness, revision coverage and final completeness. The 2 Mbit/s limit was not restrictive enough in this workload for semantic suppression to offset adaptive probing and pacing overhead. Hence adaptive backpressure is beneficial when receivers actually fall behind, not inherently faster under every network. More restrictive bandwidth rates and burstier sources could reveal the crossover point.

OPTIMISATION FINDING
Adaptive preserves finals and sharply improves receiver-backpressure latency, but incurs measurable overhead at the tested 2 Mbit/s setting. Both favorable and unfavorable outcomes are included.

7 Failures, trade-offs and limitations

7.1 What the experiments did not solve

First, slow readers still experienced substantial latency under adaptive mode (slow P95 8.18s, overall P99 6.11s). Second, under combined delay and loss, 100% of final captions arrived but only 89.49% did so within the 500ms freshness threshold, revision coverage fell to 58.05%. This exposes a timeliness-versus-completeness limitation, rather than final-state loss in the latest run. Third, adaptive scheduling adds overhead without congestion. When a client exceeds the 256-segment pending cap, WebSocket close 1013 bounds application backlog, but reconnect recovery remains limited by retained recent history.

7.2 Validity and resource bounds

The source and client population are deterministic, but latency is measured at simulated Python clients rather than browser paint. Tests exclude speech recognition, classroom Wi-Fi contention, mobile power management and clock-offset effects, and use at most 100 clients. A separate five-snapshot T3 monitor showed 10 congested clients, 390 cumulative coalesced updates and RSS of 32.8 – 33.4 MiB over 20s, this supports short-term stability, not a whole-run RSS bound. Application-level queue/history caps are explicit, but measured queue high-water marks, CPU usage and post-overload recovery remain less thoroughly characterized.

7.3 Engineering trade-offs

The design gives normal users full revision visibility when possible and switches to semantic reduction when lag is detected. Independent channels isolate healthy readers, while preserving and prioritizing finals favours user-visible correctness. Ping-based detection adds control traffic, drain/pacing can keep some provisional text delayed, bounded history necessarily discards old context. The intentionally unbounded FIFO mode demonstrates why “queue every update” cannot be the submitted overload policy, even when its no-contention P95 is slightly lower.

8 Future work / FYP direction

A future FYP could study capacity-aware semantic fan-out for lecture captioning. We would instrument queue peaks, RSS, ping lag and policy transitions, evaluate 500+ clients, synchronized reconnects, and bandwidth profiles below 2 Mbit/s, and optimize the transition between full revision delivery and final-priority operation. A classroom prototype could integrate local real-time ASR into the existing protocol and study how quickly students recover missed concepts, beyond networking-level freshness alone.

9 Conclusion

LiveRecall provides a revision-correct caption workflow, local 15/30/60-second recall, bounded state and incremental WebSocket recovery. Across the latest public T1–T5 results, every scenario achieved 100% final completeness, T2 P95 was 32.7ms. Adaptive delivery cut T3 slow-reader overall P95 by 98.62% versus coalescing while retaining all finals, although high slow-client tails and overhead under 2 Mbit/s show that semantics-aware backpressure must be assessed together with freshness, revision coverage and workload conditions.

Appendix A. Reproducibility and Source Traceability

A.1 Environment and installation

Experiments were run on Ubuntu Linux with Python 3.12, the deterministic caption_source.py and simulated WebSocket clients. The server and load clients ran in the same Ubuntu environment, Linux tc/netem or tbf impaired the loopback interface (lo). The reviewed source revision is 3ab4786139800303d65728c9fa647470292daf7a, historical CSV/JSON files do not themselves embed a tested-commit hash.

```
sudo apt update && sudo apt install -y python3 python3-venv python3-pip
iproute2 git
git clone https://github.com/idunruniroll/CS3103.git && cd CS3103
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

To launch the submitted server independently: python solution_server.py --port 8765. For automated tests, leave this server stopped: run_demo.py starts and terminates its own instance on port 8765. A leftover server causes an address-already-in-use error.

A.2 Public tests and policy comparisons

From the repository root, run the following scripts. Each public scenario and policy mode is repeated three times. Outputs are stored in experiments/data/.

```
RUNS=3 bash experiments/run_public_tests.sh       # T1-T4
RUNS=3 bash experiments/run_netem.sh              # T5 requires sudo/tc
RUNS=3 WITH_BANDWIDTH=1 bash experiments/compare_policies.sh
python experiments/summarise.py experiments/data --output
experiments/data/SUMMARY.md
```

T1: 20 clients/20 s. T2: 100 clients/40s. T3: 100 clients/60s with 10% slow readers (500ms read delay). T4: 100 clients/60s, 20% disconnecting at 15s for 5s. T5: 100 clients/40 s per impairment profile. Results are medians of three per-run statistics, not pooled percentiles.

A.3 Captured network impairment profiles

Profile	Linux tc configuration on loopback (lo)
bandwidth_2mbit	tbf: 2 Mbit/s, 4 KiB burst, 400 ms latency
delay_100ms	netem: 100 ms delay, 20 ms jitter
loss_3pct	netem: 3% loss
delay_loss	netem: 120 ms delay, 30 ms jitter, 3% loss
reorder	netem: 20 ms delay; 20% reorder; 50% correlation; gap 1

Impairments were reset between profiles. To confirm the current state after testing, run sudo tc qdisc show dev lo. The baseline output may show qdisc noqueue when loopback shaping has been removed.

A.4 Metric verification and source provenance

The public analyser reports caption age (client receive time minus source_ts), P50/P95/P99, 500 ms live freshness, generated-final freshness, unique final completeness, and live revision coverage. T3 additionally reports normal- and slow-client latency, T4 reports HISTORY recovery response timing. The source-to-client benchmark excludes browser rendering latency.

Run logs, per-run CSV/JSON records, and the generated SUMMARY.md provide audit evidence. The policy results in Section 6 were generated with identical settings per mode and were not pooled with the separate mandatory T3/T5 runs. Any rerun should record git rev-parse HEAD and git status --short alongside the machine configuration. Automated grading depends on the exact submitted code rather than this report alone.

Short monitoring evidence from a separate 100-client slow-reader run: five snapshots over 20 seconds measured 32.8–33.4 MiB RSS, observed 10 congested clients, and counted 390 cumulative coalesced updates. These snapshots show the congestion policy was exercised but are insufficient to prove memory stabilization over a longer run.

A.5 LiveRecall server architecture and per-client delivery

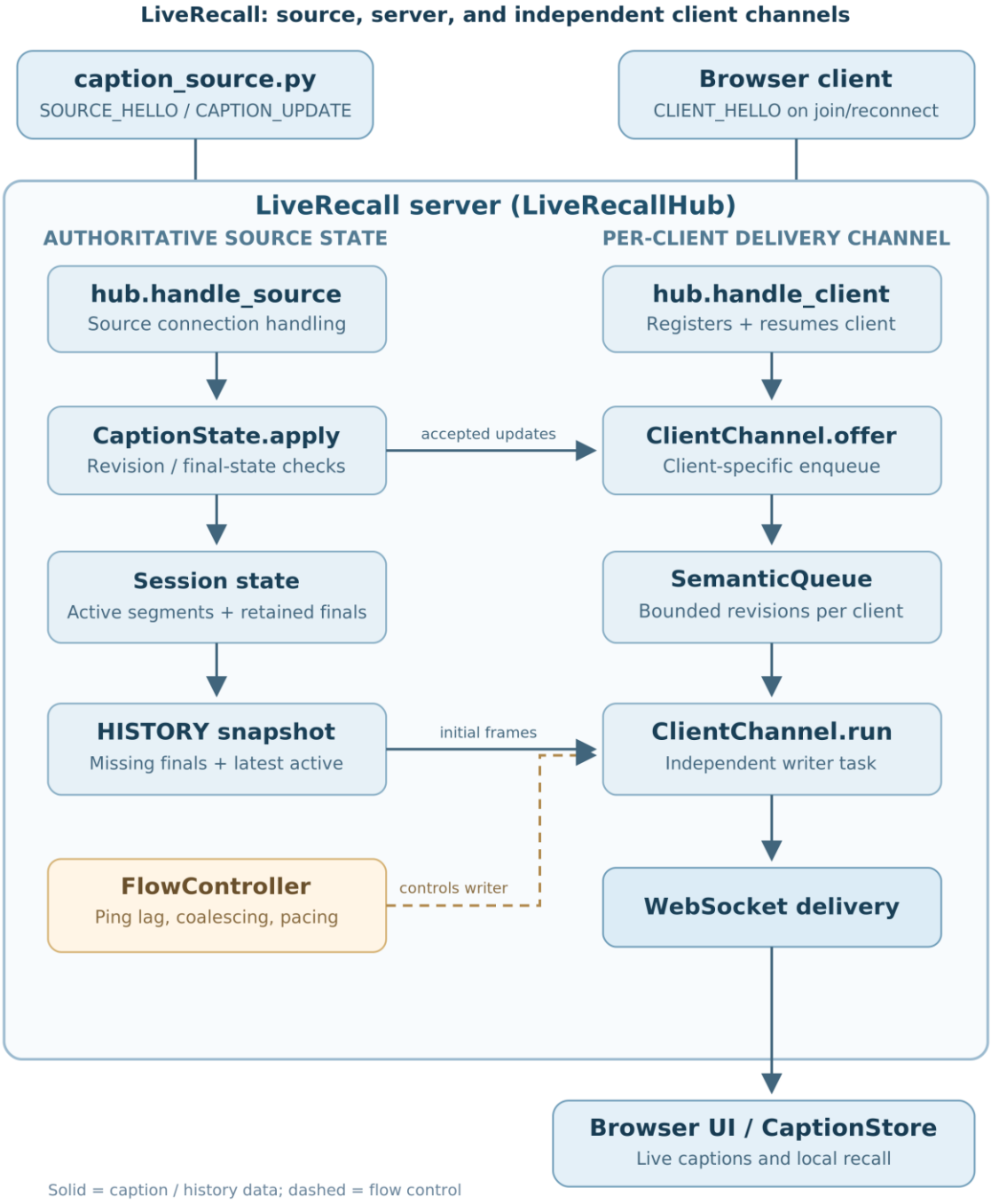


Figure A.1. The source feeds revision-aware session state, accepted updates enter independent per-client semantic queues. FlowController governs the writer's task rather than appearing in the data path. On join/reconnect, the client handler sends recent missing HISTORY and active state as initial frames before continuing live delivery. The browser and caption source are separate from the server.